

Examen de seconde session SRCS Master 1 SAR

Jonathan Lejeune

Juin 2018

2 heures – **Tout document papier interdit**
Tout appareil électronique interdit

- Le barème est sur 22 points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Lors de la correction il sera tenu compte de la simplicité et de lisibilité de vos réponses
- Pour simplifier le code, **sauf si c'est explicitement demandé** vous ne tiendrez pas compte de la gestion des exceptions ainsi que des `import`.
- Il vous est conseillé de lire attentivement les exercices entièrement avant de composer.

Exercice 1 – Questions de cours (8 points)

Question 1 1 point

Donnez un avantage et un inconvénient des appels de procédure distante par rapport aux sockets ?

Question 2 1 point

Pourquoi parle-t-on d'architecture 3-tiers dans le modèle client-serveur ?

Question 3 1 point

Définir squelette et souche.

Soient les classes Java A et B suivantes :

```
public class A {  
    public synchronized void f(B b){  
        b.h(this);  
    }  
    public synchronized void g(){  
        System.out.println("Bonjour");  
    }  
}  
  
public class B{  
    public void h(A a){  
        a.g();  
    }  
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Question 4 1 point

On suppose que l'on dispose d'une instance de A référencée par une variable `a` et d'une instance de B référencée par une variable `b`. Peut-on avoir un interblocage si on fait appel à la méthode `a.f(b)` ? **Justifiez**

Question 5 1 point

Même question si on considère que les classes A et B sont des classes d'implémentation de services RMI et que `a` et `b` sont des références distantes. **Justifiez**

Question 6 1 point

Pour une interface CORBA `Foo`, quelle est l'utilité de la classe `FooHelper` ?

Question 7 1½ points

Dans chacun des trois intergiciels abordés (RMI, CORBA, EJB), comment faire pour appeler une méthode distante de manière asynchrone ?

Question 8 ½ point

Qu'est ce qu'un EJB entity ?

Exercice 2 – Table associative observable en Java RMI

L'API standard Java offre l'interface `Map<K,V>` qui permet d'associer à un objet clé de type `K` un objet valeur de type `V`. Vous trouverez en annexe de ce document un extrait de cette interface.

L'objectif de cet exercice est de fournir dans un premier temps une version simplifiée de cette interface accessible à distance via RMI et dans un deuxième temps d'étendre ces fonctionnalités pour que les clients puissent être notifiés des changements de valeur sur une clé donnée.

Nous proposons donc l'interface `RemoteMap` suivante :

```
public interface RemoteMap<K, V> {
    void put(K key, V value) ;
    V get(K key) ;
    boolean containsKeys(K key) ;
    V remove(K key);
}
```

1
2
3
4
5
6
7
8

Question 1 1½ points

Modifiez l'interface `RemoteMap` pour qu'elle puisse être utilisée en tant que service RMI. Vous assurerez que les paramètres `K` et `V` sont passés par copie.

Question 2 2 points

Donnez la code de la classe `RemoteMapImpl` qui implante l'interface `RemoteMap`.

On souhaite maintenant que les clients puissent s'abonner à une clé donnée de la map et être notifiés pour chaque opération sur cette clé (création, modification de la valeur, suppression). La notification se fera grâce à une interface de rappel qu'on appellera **Watcher**. Pour ceci on souhaite proposer les opérations suivantes :

- **setWatcher** : permet d'enregistrer sur une clé de type générique K, un **Watcher**. La méthode renverra un entier qui permet d'identifier le **Watcher** par rapport à la map.
- **unsetWatcher** : qui permet de désabonner un **Watcher** d'une clé. En paramètre, on passera la clé concernée et l'identifiant interne renvoyé préalablement par **setWatcher** pour identifier le **Watcher** qui souhaite se désabonner. Cette méthode renverra faux si la clé associée ou l'identifiant du **Watcher** n'existe pas, vrai si le désabonnement a bien été effectué.

Un **Watcher** doit pouvoir être notifié de :

- la création de la clé (si celle-ci n'existait pas encore au moment de l'enregistrement du **Watcher**)
- le changement de la valeur associée à la clé
- la suppression de la clé.

L'instance d'un **Watcher** peut très bien être abonnée à plusieurs clés. C'est ainsi que l'on souhaitera transmettre au **Watcher** pour chaque événement, la clé concernée de type générique K. On transmettra également la valeur associée à la clé de type générique V : si l'événement est une création ou une modification on transmettra la nouvelle valeur, si c'est une suppression on transmettra la dernière valeur associée.

Question 3 1½ points

Écrire l'interface **Watcher**.

Question 4 1½ points

Écrire l'interface **WatchedRemoteMap** qui étend l'interface **RemoteMap** et qui offre les méthodes **setWatcher** et **unsetWatcher**.

Question 5 3 points

Écrire la classe d'implémentation **WatchedRemoteMapImpl** qui implante l'interface **WatchedRemoteMap**. Vous étendrez la classe **RemoteMapImpl** et vous redéfinirez les méthodes lorsque ceci est nécessaire. Vous veillerez à protéger les données partagées des accès concurrents.

Question 6 1½ points

Écrire la classe **ServeurMap** qui permet de déployer le service RMI **WatchedRemoteMap<String,String>** et d'enregistrer ce service sous le nom de **"remotemap"** dans le registry.

Question 7 3 points

Écrire le code des classes permettant à un client de s'abonner à la clé **"srcs"** sur la **RemoteMap** déployée à la question précédente. Le thread principal sera mis en attente tant que la clé **"srcs"** n'est pas supprimée. Dès que la clé est supprimée on réveillera le thread principal qui désabonnera le client et terminera correctement le programme.

Annexe

Extrait de l'interface Map de l'API Java

```
public interface Map<K,V> {  
  
    int size(); //retourne le nombre de clés  
    boolean isEmpty(); //teste si la map est vide  
    boolean containsKey(Object key); //teste si une clé est présente dans la map  
    boolean containsValue(Object value); //teste si il existe une clé associé à la valeur  
        value  
  
    V get(Object key); //obtenir la valeur associée à une clé, retourne null si la clé n'  
        existe pas dans la map  
  
    V put(K key, V value); //associe une valeur à une clé, si la clé existe déjà alors l'  
        ancienne valeur est remplacée par la nouvelle valeur  
  
    V remove(Object key); //même spécification que get mais supprime en plus la clé de la  
        map  
  
    void clear(); //efface toutes les clés de la map  
  
    Set<K> keySet(); //retourne l'ensemble des clés contenues dans la map  
  
    Collection<V> values(); //retourne l'ensemble des valeurs contenues dans la map  
  
    Set<Map.Entry<K, V>> entrySet(); //retourne un ensemble de couples dont le premier élé  
        ment est une clé et le deuxième élément la valeur associée. Le type Map.entry  
        offre une méthode getKey() et une méthode getValue() qui retournent  
        respectivement la clé du couple et la valeur associée.  
  
    ...  
}
```